

FISPACT-II workshop

Nuclear Data & FISPACT-II

1. Loading Nuclear data
 - i. Standard FISPACT-II files approach
 - ii. Python FISPACT-API approach
2. Probing Nuclear data
 - i. Standard FISPACT-II files approach
 - ii. Python FISPACT-API approach
3. Example highlighting importance of nuclear data choice

Loading Nuclear Data

Standard FISPACT-II: the *files* file

- Which Nuclear Data is read by FISPACT-II is controlled by the *files* file.
- The *files* file contains the directory paths to the Nuclear Data files on the machine which is running FISPACT-II.
- Index strings tell FISPACT-II which data is in which directory/file
- Mistakes in the *files* file are a very common source of error for FISPACT-II users. Be sure to check your *files* file before you run your *input* file

key followed by path, # = comment

```
# index of nuclides to be included
ind_nuc /opt/fispact/nuclear_data/TENDL2017data/tendl17_decay12_index

# Library cross section data
xs_endf /opt/fispact/nuclear_data/TENDL2017data/tal2017-n/gxs-709

# Library probability tables for self-shielding
prob_tab /opt/fispact/nuclear_data/TENDL2017data/tal2017-n/tp-709-294

#fluxes
fluxes fluxes

# Library decay data
dk_endf /opt/fispact/nuclear_data/decay/decay_2012

# Library fission data
fy_endf /opt/fispact/nuclear_data/ENDFB71data/endfb71-n/endfb71nfy
sf_endf /opt/fispact/nuclear_data/ENDFB71data/endfb71-n/endfb71sfy

# Library regulatory data
hazards /opt/fispact/nuclear_data/decay/hazards_2012
clear /opt/fispact/nuclear_data/decay/clear_2012
a2data /opt/fispact/nuclear_data/decay/a2_2012

# gamma attenuation data
absorp /opt/fispact/nuclear_data/decay/abs_2012

# collapsed cross section data (in and out)
collapxi COLLAPX
collapxo COLLAPX

# condensed decay and fission data (in and out)
arrayx ARRAYX
```

What Nuclear data does a *files* file point to? (1)

ind_nuc

Nuclide index. The list of nuclides known to the simulation.

Here index file for decay2012 covering nuclides in TENDL2017

Note: different keys for EAF-style libraries, see manual

```
# index of nuclides to be included
ind_nuc /opt/fispact/nuclear_data/TENDL2017data/tendl17_decay12_index

# Library cross section data
xs_endf /opt/fispact/nuclear_data/TENDL2017data/tal2017-n/gxs-709

# Library probability tables for self-shielding
prob_tab /opt/fispact/nuclear_data/TENDL2017data/tal2017-n/tp-709-294

#fluxes
fluxes fluxes

# Library decay data
dk_endf /opt/fispact/nuclear_data/decay/decay_2012
```

xs_endf

Nuclide cross section data in ENDF format.

Determine which reactions are known to FISPACT-II. Here loading TENDL2017

prob_tab

Nuclide probability tables used for self-shielding calculations.

Here loading those from TENDL2017

dk_endf

Nuclide decay data. Source of radiological observables

Here loading decay 2012.

What Nuclear data does a *files* file point to? (2)

fy_endf - induced
sy_endf - spontaneous

Fission yield data. Induced and spontaneous data.

Here loading from ENDF/BVII

hazards

Biological Hazards data used in inhalation and ingestion dose.

Here from decay2012

clear

Clearance data used find the clearance index of a sample

Here from decay2012

```
# Library fission data
fy_endf /opt/fispact/nuclear_data/ENDFB71data/endfb71-n/endfb71nfy
sf_endf /opt/fispact/nuclear_data/ENDFB71data/endfb71-n/endfb71sfy

# Library regulatory data
hazards /opt/fispact/nuclear_data/decay/hazards_2012
clear /opt/fispact/nuclear_data/decay/clear_2012
a2data /opt/fispact/nuclear_data/decay/a2_2012

# gamma attenuation data
absorp /opt/fispact/nuclear_data/decay/abs_2012
```

Gamma attenuation data using when calculated dose rates.

Here from decay2012

absorp

a2data

A2 transport data.

Here from decay2012

Changing Nuclear Data

- To run a simulation with a different set of Nuclear Data a user must construct a new or edit an existing *files* file.
- Simply change the path to point to the desired Nuclear data. Log file will detail which nuclear data was loaded for future reference.
- Nuclear data can be chosen in any combination, but users should always be wary. Testing surprising results with other nuclear data is encouraged.

TENDL2017, decay2012

TENDL2019, decay2020

```
# index of nuclides to be included
ind_nuc /opt/fispact/nuclear_data/TENDL2017data/tendl17_decay12_index

# Library cross section data
xs_endf /opt/fispact/nuclear_data/TENDL2017data/tal2017-n/gxs-709

# Library probability tables for self-shielding
prob_tab /opt/fispact/nuclear_data/TENDL2017data/tal2017-n/tp-709-294

#fluxes
fluxes fluxes

# Library decay data
dk_endf /opt/fispact/nuclear_data/decay/decay_2012
```

```
# index of nuclides to be included
ind_nuc /opt/fispact/nuclear_data/decay2020/decay_2020_index.txt

# Library cross section data
xs_endf /opt/fispact/nuclear_data/tendl19data709/gendf-709

# Library probability tables for self-shielding
prob_tab /opt/fispact/nuclear_data/tendl19data709/tp-709-294

#fluxes
fluxes fluxes

# Library decay data
dk_endf /opt/fispact/nuclear_data/decay2020/decay_2020
```

Note: Users must point to where the nuclear data is store on their device. This will likely not be */opt/fispact/*

Pyfispact: FISPACT-II Python API

- Nuclear data loading is handled by the **NuclearDataReader** class.
- This populates the **NuclearData** class with the desired nuclear data.
- As with the files file, the **NuclearDataReader** takes in **keys** and file **paths**.

Keys match *files* file equivalents.

e.g. **ND_INC_NUC_KEY** = **ind_nuc**

Paths are identical to those given in the *files* file

```
# create NuclearDataReader
ndr = pf.io.NuclearDataReader(m)

# set paths o Nuclear data files
ndr.setpath(pf.io.ND_IND_NUC_KEY(), "/opt/fispact/nuclear_data/TENDL2017data/tendl17_decay12_index")
ndr.setpath(pf.io.ND_XS_ENDF_KEY(), "/opt/fispact/nuclear_data/TENDL2017data/tal2017-n/gxs-709")
ndr.setpath(pf.io.ND_PROB_TAB_KEY(), "/opt/fispact/nuclear_data/TENDL2017data/tal2017-n/tp-709-294")
ndr.setpath(pf.io.ND_FY_ENDF_KEY(), "/opt/fispact/nuclear_data/ENDFB71data/endfb71-n/endfb71nfy")
ndr.setpath(pf.io.ND_SF_ENDF_KEY(), "/opt/fispact/nuclear_data/ENDFB71data/endfb71-n/endfb71sfy")
ndr.setpath(pf.io.ND_DK_ENDF_KEY(), "/opt/fispact/nuclear_data/decay/decay_2012")
ndr.setpath(pf.io.ND_HAZARDS_KEY(), "/opt/fispact/nuclear_data/decay/hazards_2012")
ndr.setpath(pf.io.ND_CLEAR_KEY(), "/opt/fispact/nuclear_data/decay/clear_2012")
ndr.setpath(pf.io.ND_A2DATA_KEY(), "/opt/fispact/nuclear_data/decay/a2_2012")
ndr.setpath(pf.io.ND_ABSORP_KEY(), "/opt/fispact/nuclear_data/decay/abs_2012")

# create Nucleardata object
nd = pf.NuclearData(m)

# load nuclear data
ndr.load(nd, op=loadfunc)
```

n.b. **pf** is the **pyfispact** module, **m** is the **Monitor** (error handling). See yesterdays demonstration.

Nuclear data is loaded in to the **NuclearData** object with the *load* function.

The loaded data is identical to that used by the standard version.

Pyfispact: What are we looking at?

Initialise the **NuclearDataReader** object

Set the paths to the various data

Initialise the **NuclearData** object. This stores the data and is input into the FISPACT-II engine

```
# create NuclearDataReader
ndr = pf.io.NuclearDataReader(m)
# set paths o Nuclear data files
ndr.setpath(pf.io.ND_IND_NUC_KEY(), "/opt/fispact/nuclear_data/TENDL2017data/tendl17_decay12_index")
ndr.setpath(pf.io.ND_XS_ENDF_KEY(), "/opt/fispact/nuclear_data/TENDL2017data/tal2017-n/gxs-709")
ndr.setpath(pf.io.ND_PROB_TAB_KEY(), "/opt/fispact/nuclear_data/TENDL2017data/tal2017-n/tp-709-294")
ndr.setpath(pf.io.ND_FY_ENDF_KEY(), "/opt/fispact/nuclear_data/ENDFB71data/endfb71-n/endfb71nfy")
ndr.setpath(pf.io.ND_SF_ENDF_KEY(), "/opt/fispact/nuclear_data/ENDFB71data/endfb71-n/endfb71sfy")
ndr.setpath(pf.io.ND_DK_ENDF_KEY(), "/opt/fispact/nuclear_data/decay/decay_2012")
ndr.setpath(pf.io.ND_HAZARDS_KEY(), "/opt/fispact/nuclear_data/decay/hazards_2012")
ndr.setpath(pf.io.ND_CLEAR_KEY(), "/opt/fispact/nuclear_data/decay/clear_2012")
ndr.setpath(pf.io.ND_A2DATA_KEY(), "/opt/fispact/nuclear_data/decay/a2_2012")
ndr.setpath(pf.io.ND_ABSORP_KEY(), "/opt/fispact/nuclear_data/decay/abs_2012")
# create Nucleardata object
nd = pf.NuclearData(m)
# load nuclear data
ndr.load(nd, op=loadfunc)
```

Load the data, populating the **NuclearData** object. Optionally including a callback function, printing the progress of the load to screen. Analogous to **MONITOR** keyword.

```
def loadfunc(k, p, index, total):
    print(" [{} / {}] Reading {}: {}".format(index, total, k, p), end="\r")
    sys.stdout.write("\033[K")
```

Different Nuclear Data with Pyfispact

- In the same manner as with the *files* file, the paths are altered to load different data.
- One advantage of the API approach is that multiple **NuclearData** objects can be populated. Allowing quick comparison between Nuclear Data sets.
- A user can define a function which will load a given set of Nuclear Data for each combination desired.

TENDL2017, decay2012

```
def load_TENDL17decay2012():
    # create NuclearDataReader
    ndr = pf.io.NuclearDataReader(m)
    # set paths o Nuclear data files
    ndr.setpath(pf.io.ND_IND_NUC_KEY(), "/opt/fispact/nuclear_data/TENDL2017data/tendl17_decay12_index")
    ndr.setpath(pf.io.ND_XS_ENDF_KEY(), "/opt/fispact/nuclear_data/TENDL2017data/tal2017-n/gxs-709")
    ndr.setpath(pf.io.ND_PROB_TAB_KEY(), "/opt/fispact/nuclear_data/TENDL2017data/tal2017-n/tp-709-294")
    ndr.setpath(pf.io.ND_FY_ENDF_KEY(), "/opt/fispact/nuclear_data/ENDFB71data/endfb71-n/endfb71nfy")
    ndr.setpath(pf.io.ND_SF_ENDF_KEY(), "/opt/fispact/nuclear_data/ENDFB71data/endfb71-n/endfb71sfy")
    ndr.setpath(pf.io.ND_DK_ENDF_KEY(), "/opt/fispact/nuclear_data/decay/decay_2012")
    ndr.setpath(pf.io.ND_HAZARDS_KEY(), "/opt/fispact/nuclear_data/decay/hazards_2012")
    ndr.setpath(pf.io.ND_CLEAR_KEY(), "/opt/fispact/nuclear_data/decay/clear_2012")
    ndr.setpath(pf.io.ND_A2DATA_KEY(), "/opt/fispact/nuclear_data/decay/a2_2012")
    ndr.setpath(pf.io.ND_ABSORP_KEY(), "/opt/fispact/nuclear_data/decay/abs_2012")
    # create Nucleardata object
    nd = pf.NuclearData(m)
    # load nuclear data
    ndr.load(nd, op=loadfunc)
    return nd
```

TENDL2019, decay2020

```
def load_TENDL19decay2020():
    # create NuclearDataReader
    ndr = pf.io.NuclearDataReader(m)
    # set paths o Nuclear data file
    ndr.setpath(pf.io.ND_IND_NUC_KEY(), "/opt/fispact/nuclear_data/decay2020/decay_2020_index.txt")
    ndr.setpath(pf.io.ND_XS_ENDF_KEY(), "/opt/fispact/nuclear_data/tendl19data709/gendf-709")
    ndr.setpath(pf.io.ND_PROB_TAB_KEY(), "/opt/fispact/nuclear_data/tendl19data709/tp-709-294")
    ndr.setpath(pf.io.ND_FY_ENDF_KEY(), "/opt/fispact/nuclear_data/ENDFB71data/endfb71-n/endfb71nfy")
    ndr.setpath(pf.io.ND_SF_ENDF_KEY(), "/opt/fispact/nuclear_data/ENDFB71data/endfb71-n/endfb71sfy")
    ndr.setpath(pf.io.ND_DK_ENDF_KEY(), "/opt/fispact/nuclear_data/decay2020/decay_2020")
    ndr.setpath(pf.io.ND_HAZARDS_KEY(), "/opt/fispact/nuclear_data/decay/hazards_2012")
    ndr.setpath(pf.io.ND_CLEAR_KEY(), "/opt/fispact/nuclear_data/decay/clear_2012")
    ndr.setpath(pf.io.ND_A2DATA_KEY(), "/opt/fispact/nuclear_data/decay/a2_2012")
    ndr.setpath(pf.io.ND_ABSORP_KEY(), "/opt/fispact/nuclear_data/decay/abs_2012")
    # create Nucleardata object
    nd = pf.NuclearData(m)
    # load nuclear data
    ndr.load(nd, op=loadfunc)
    return nd
```



Probing Nuclear Data

Standard FISPACT-II: PRINTLIB keyword

- The **PRINTLIB** keyword will print details of eth loaded nuclear data to the output file.
- PRINTLIB** takes an integer argument which will determine what is output. Complete details of this are given in the manual and on the wiki.

			PERCENTAGE			BRANCHING			RATIOS		
1											
H	3	(b-)	He	3	1.000E+02	H	4	(n)	H	3	1.000E+02
H	6	(3n)	H	3	5.000E+01	H	7	(2n)	H	5	1.000E+02
He	7	(n)	He	6	1.000E+02	He	8	(b-)	Li	8	8.800E+01
He	10	(2n)	He	8	1.000E+02	Li	4	(p)	He	3	1.000E+02
Li	9	(b-)	Be	9	5.050E+01	Li	9	(b-n)	Be	8	4.950E+01
Li	11	(b-n)	Be	10	8.490E+01	Li	11	(b-2n)	Be	9	4.100E+00
Be	5	(p)	Li	4	1.000E+02	Be	6	(pp)	He	4	1.000E+02
Be	10	(b-)	B	10	1.000E+02	Be	11	(b-)	B	11	9.700E+01
Be	13	(n)	Be	12	1.000E+02	Be	14	(b-)	B	14	1.561E-15
Be	14	(b-3n)	B	11	2.000E-01	Be	15	(n)	Be	14	1.000E+02
B	7	(p)	Be	6	1.000E+02	B	8	(b+a)	He	4	1.000E+02
B	12	(b-a)	Be	8	1.580E+00	B	13	(b-)	C	13	9.972E+01
B	15	(b-)	C	15	6.000E+00	B	15	(b-n)	C	14	9.360E+01
B	17	(b-)	C	17	2.250E+01	B	17	(b-n)	C	16	6.300E+01
B	18	(n)	B	17	1.000E+02	B	19	(b-n)	C	18	7.500E+01
C	9	(b+)	B	9	6.000E+01	C	9	(b+p)	Be	8	2.300E+01
C	11	(b+)	B	11	1.000E+02	C	14	(b-)	N	14	1.000E+02
C	16	(b-n)	N	15	9.790E+01	C	17	(b-)	N	17	7.160E+01
C	18	(b-n)	N	17	3.150E+01	C	19	(b-)	N	19	4.600E+01
C	20	(b-)	N	20	2.800E+01	C	20	(b-n)	N	19	7.200E+01
C	22	(b-n)	N	21	9.900E+01	N	10	(p)	C	9	1.000E+02
N	13	(b+)	C	13	1.000E+02	N	16	(b-)	O	16	1.000E+02
N	17	(b-n)	O	16	9.500E+01	N	17	(b-a)	C	13	2.500E-03
N	19	(b-n)	O	18	5.460E+01	N	20	(b-)	O	20	4.300E+01
N	21	(b-n)	O	20	8.100E+01	N	22	(b-)	O	22	6.500E+01
						H	5	(2n)	H	3	1.000E+02
						He	5	(n)	He	4	1.000E+02
						He	8	(b-n)	Li	7	1.200E+01
						Li	5	(p)	He	4	1.000E+02
						Li	10	(n)	Li	9	1.000E+02
						Li	11	(b-3n)	Be	8	1.900E+00
						Be	7	(b+)	Li	7	1.000E+02
						Be	11	(b-a)	Li	7	3.000E+00
						Be	14	(b-n)	B	13	9.800E+01
						Be	16	(2n)	Be	14	1.000E+02
						B	9	(p)	Be	8	1.000E+02
						B	13	(b-n)	C	12	2.760E-01
						B	15	(b-2n)	C	13	4.000E-01
						B	17	(b-2n)	C	15	1.100E+01
						B	19	(b-2n)	C	17	2.500E+01
						C	9	(b+a)	Li	5	1.700E+01
						C	15	(b-)	N	15	1.000E+02
						C	17	(b-n)	N	16	2.840E+01
						C	19	(b-n)	N	18	4.700E+01
						C	21	(n)	C	20	1.000E+02
						N	11	(p)	C	10	1.000E+02
						N	16	(b-a)	C	12	1.200E-03
						N	18	(b-)	O	18	1.000E+02
						N	20	(b-n)	O	19	5.700E+01
						N	22	(b-n)	O	21	3.500E+01
						H	6	(n)	H	5	5.000E+01
						He	6	(b-)	Li	6	1.000E+02
						He	9	(n)	He	8	1.000E+02
						Li	8	(b-a)	He	4	1.000E+02
						Li	11	(b-)	Be	11	9.100E+00
						Li	12	(n)	Li	11	1.000E+02
						Be	8	(a)	He	4	1.000E+02
						Be	12	(b-)	B	12	1.000E+02
						Be	14	(b-2n)	B	12	1.800E+00
						B	6	(pp)	Li	4	1.000E+02
						B	12	(b-)	C	12	9.842E+01
						B	14	(b-)	C	14	1.000E+02
						B	16	(n)	B	15	1.000E+02
						B	17	(b-3n)	C	14	3.500E+00
						C	8	(pp)	Be	6	1.000E+02
						C	10	(b+)	B	10	1.000E+02
						C	16	(b-)	N	16	2.100E+00
						C	18	(b-)	N	18	6.850E+01
						C	19	(b-2n)	N	17	7.000E+00
						C	22	(b-)	N	22	1.000E+00
						N	12	(b+)	C	12	1.000E+02
						N	17	(b-)	O	17	5.000E+00
						N	19	(b-)	O	19	4.540E+01
						N	21	(b-)	O	21	1.900E+01
						N	23	(b-)	O	23	2.000E+01

Example of **PRINTLIB** output displaying the decay types and branching ratios for nuclides within deacy2020.

This can be achieved, along with other data by including:
PRINTLIB 2
In a FISPACT-II input file

Standard FISPACT-II: EXTRACTXS

- Extract the group wise cross section from nuclear data.
- Available as a separate command line tool (***extract_xs_endf***) or a keyword **EXTRACTXS** (new for v5).
- Where a specific product/daughter exists it must be given.
- Otherwise (e.g. total) a string must be given and is used for file naming (***target_MT_product.xls***)
- Produced output contains the xs, flux and reaction rates and uncertainties where available

Target MT number Product /label

```
EXTRACTXS Zr90 103 Y90m
EXTRACTXS N14 103 C14
EXTRACTXS Fe56 1 TOTAL
EXTRACTXS Ca40 3 NONEL
EXTRACTXS Nb93 5 OTHER
EXTRACTXS U235 18 FISS
```

Input file

	En-low	En-high	flux	flux-unc	gxs	gxs-unc	greac-rate	cum-rate
1.00000E-05	1.04713E-05	7.18913E-05	0.00000E+00	9.08566E+01	0.00000E+00	6.53180E-03	6.53180E-03	
1.04713E-05	1.09648E-05	7.18913E-05	0.00000E+00	8.87885E+01	0.00000E+00	6.38312E-03	1.29149E-02	
1.09648E-05	1.14815E-05	7.18913E-05	0.00000E+00	8.67651E+01	0.00000E+00	6.23765E-03	1.91526E-02	
1.14815E-05	1.20226E-05	7.18889E-05	0.00000E+00	8.47910E+01	0.00000E+00	6.09553E-03	2.52481E-02	
1.20226E-05	1.25893E-05	7.18913E-05	0.00000E+00	8.28609E+01	0.00000E+00	5.95697E-03	3.12051E-02	
1.25893E-05	1.31826E-05	7.18913E-05	0.00000E+00	8.09760E+01	0.00000E+00	5.82146E-03	3.70265E-02	
1.31826E-05	1.38038E-05	7.18913E-05	0.00000E+00	7.91309E+01	0.00000E+00	5.68882E-03	4.27154E-02	
1.38038E-05	1.44544E-05	7.18913E-05	0.00000E+00	7.73319E+01	0.00000E+00	5.55949E-03	4.82748E-02	
1.44544E-05	1.51255E-05	7.18913E-05	0.00000E+00	7.55705E+01	0.00000E+00	5.43285E-03	5.37077E-02	

EXTRACTXS output: N14_103_C14.xls

Pyfispact: Query the NuclearData object

- The FISPACT-II API includes a number of methods which can probe the nuclear data object. Full details are given in the API manual and code reference sheets.
- These allow users to develop their own tools for Nuclear Data studies.

Example: a function which prints all nuclides with any photon spectra and how many lines are present

```
def get_nrofphotonlines(nd):
    decaynuclides = nd.getdecayzais()
    for i in range(len(decaynuclides)):
        if not nd.getdecayisstable(i):
            nrofspectra = nd.getdecaynrofspectrumtypes(i)
            nrofgammlines = 0
            nrofxyraylines = 0
            for j in range(nrofspectra):
                spectype = nd.getdecayspectrumtype(i, j)
                if spectype == pf.SPECTRUM_TYPE_GAMMA():
                    nrofgammlines = nd.getdecayspectrumnroflines(i, j)
                if spectype == pf.SPECTRUM_TYPE_X_RAY():
                    nrofxyraylines = nd.getdecayspectrumnroflines(i, j)

            if (nrofxyraylines + nrofgammlines) > 0:
                print("{}:^16} Nr of gamma lines : {:4d}   Nr of xray lines : {:4d}"
                      .format(pf.util.nuclide_from_zai(m, decaynuclides[i]), nrofgammlines, nrofxyraylines))
```

NuclearData methods

This function, for each nuclide in the decay data:

1. Checks if its stable
2. If not, get the number of decay spectra available
3. Checks if gamma or xray are present
4. If so, gets the number of lines and prints to screen

Fe52m	Nr of gamma lines :	6	Nr of xray lines :	5
Fe53	Nr of gamma lines :	10	Nr of xray lines :	4
Fe53m	Nr of gamma lines :	6	Nr of xray lines :	0
Fe55	Nr of gamma lines :	1	Nr of xray lines :	4
Fe59	Nr of gamma lines :	7	Nr of xray lines :	4
Fe61	Nr of gamma lines :	48	Nr of xray lines :	0
Fe62	Nr of gamma lines :	1	Nr of xray lines :	0
Fe63	Nr of gamma lines :	19	Nr of xray lines :	3
Fe64	Nr of gamma lines :	2	Nr of xray lines :	0
Fe65m	Nr of gamma lines :	11	Nr of xray lines :	0
Co53	Nr of gamma lines :	1	Nr of xray lines :	5
Co54	Nr of gamma lines :	1	Nr of xray lines :	5
Co54m	Nr of gamma lines :	3	Nr of xray lines :	5
Co55	Nr of gamma lines :	23	Nr of xray lines :	4
Co56	Nr of gamma lines :	46	Nr of xray lines :	5
Co57	Nr of gamma lines :	10	Nr of xray lines :	4
Co58	Nr of gamma lines :	3	Nr of xray lines :	5
Co58m	Nr of gamma lines :	1	Nr of xray lines :	4
Co60	Nr of gamma lines :	6	Nr of xray lines :	4
Co60m	Nr of gamma lines :	4	Nr of xray lines :	4
Co61	Nr of gamma lines :	3	Nr of xray lines :	4
Co62	Nr of gamma lines :	13	Nr of xray lines :	4

Data from
decay2020

Sample Output:

Example showing the importance of Nuclear Data

Example: impact of choice of Nuclear Data

- In this example 1kg of Osmium is irradiated for 10 minutes with a mono-energetic 10 MeV 1×10^{12} n/cm²/s neutron flux.
- The decay heat will be found at 1 minute intervals for 1 hour after irradiation.
- This simulation will be run using JEFF3.3, ENDFB/VIII and TENDL2021 and the results compared.
- These calculations will the decay libraries supplied with JEFF3.3 and ENDFB/VIII for those runs. Decay2012 will be used with TENDL2022.

Using Standard FISPACT-II

- In order to compare multiple nuclear data sets a *files* file for each set of data needs to be created.
- Only a single input file and flux file are needed as which *files* file to use can be specified on the command line.
- Each calculation will need to be run separately. They can be executed in sequence using a shell script.

Lets look at this example

Using Pyfispact

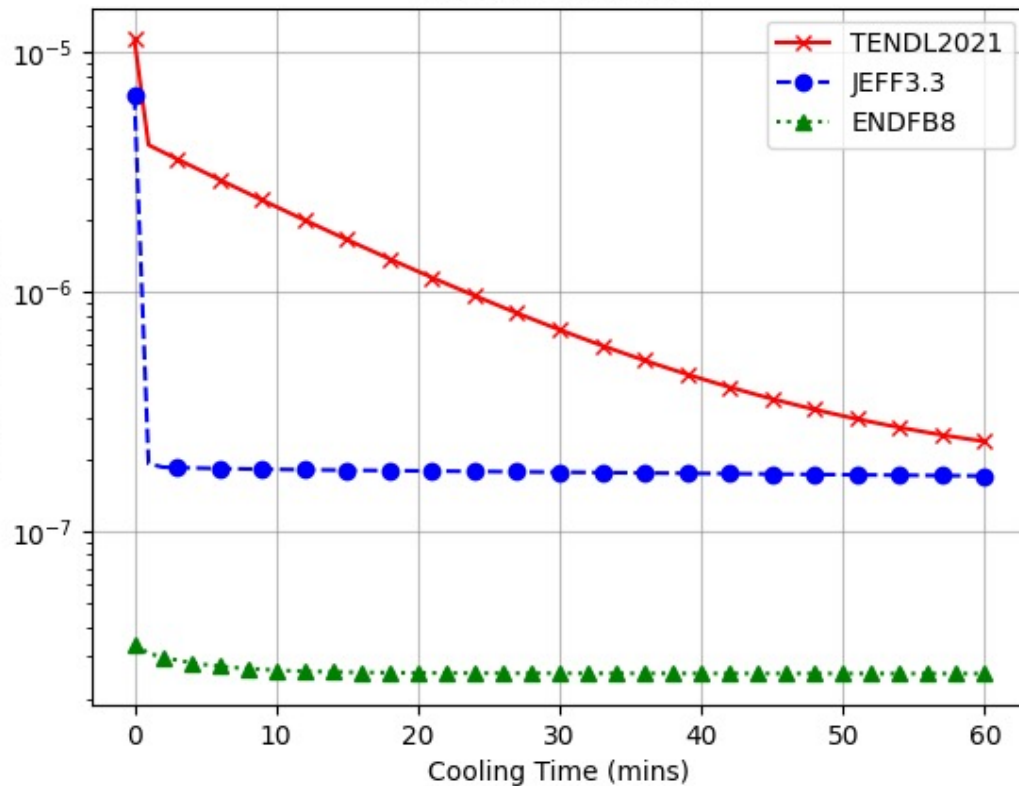
- As shown previously, separate nuclear data loading routine can be simply created for each set to be studies.
- These can be called in sequence within the same script to load the required data.
- Each simulation can also be performed in the same script, alongside analysis and plotting.

Lets look at this example

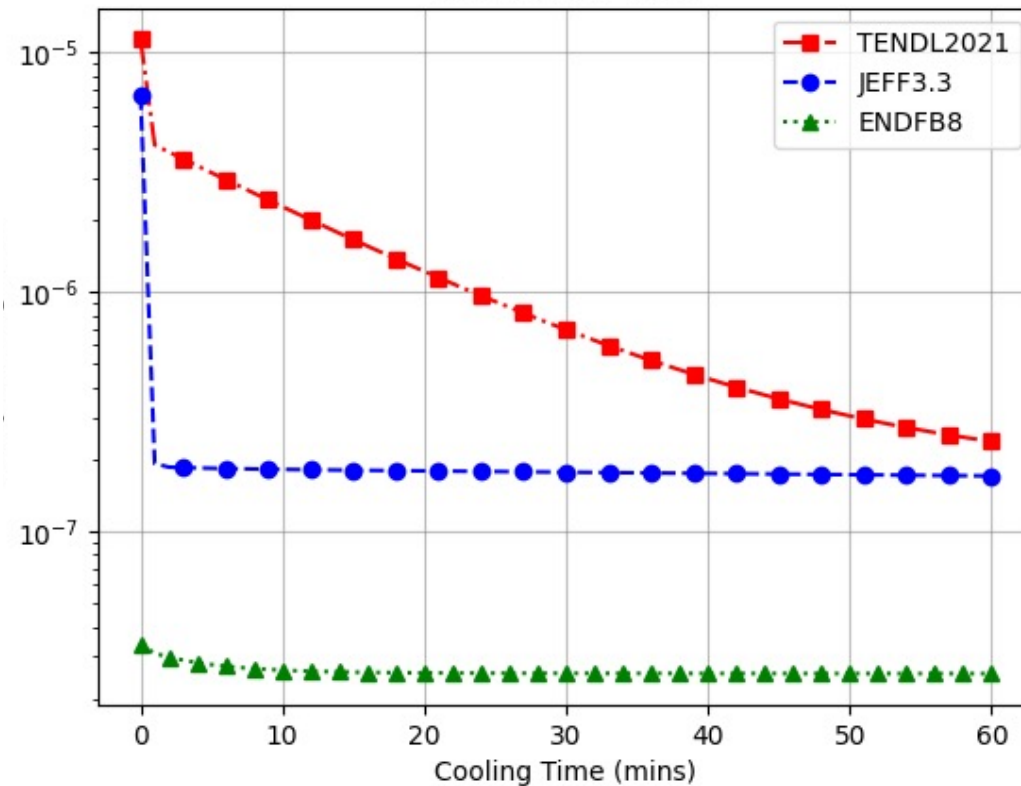
Results using both approaches

- The results using the standard approach and API agree with each other.
- The choice of Nuclear Data Libraries can have a significant impact on results.
- Differences seen here are a result of differences in cross sections and the inclusion (or lack of) isomeric states.
- Comparing Nuclear Data is straight forward with FISPACT-II

Os Decay Heating with different Nuclear Data using standard FISPACT-II



Os Decay Heating with different Nuclear Data using the FISPACT-II API



Conclusions

- Some nuclear data will match, others will not.
- Just because nuclear data libraries agree with each other does **not** imply they are the most physically accurate.
- Which nuclear data is the most *correct* can only be concluded via comparison with experiment.
 - See decay heating V&V presentation next week.
- Important users are aware how the choice of nuclear data can affect their results.
- If in doubt users should **run a simulation with multiple nuclear data sets and compare outputs.**
- **Included in the example are inputs using TENDL2017. What does this library predict for the Os irradiation?**